Question 1: Two-Dimensional Perceptron

[1]:
```
### Training with manually updating W with "Backward" ###

import torch
#from torch.autograd import Variable
import torch.nn as nn
import torch.nn.functional as F

import torch.optim as optim




data = [(1.0,2.1,3.0), (2.0, 3.5, 6.0), (3.0, 3.0, 9.0), (4.0, 2.1, 12.0), (5.0, 7.2, 15.0), (6.0, 10.1,
↪  18.0)]


class Net(nn.Module):
    def __init__(self):
        torch.manual_seed(0)
        super(Net, self).__init__()
        self.fc1 = nn.Linear(2, 1, bias=False)

    def forward(self, x):
        return self.fc1(x)

net = Net()

print(net)
print(list(net.parameters()))

#input = torch.randn(1)
#out = net(input)

#def criterion(out, label):
#    return (label - out)**2
criterion = nn.MSELoss()


optimizer = optim.SGD(net.parameters(), lr=0.01, momentum=0.5)
#optimizer = optim.Adam(net.parameters(), lr=0.005)

# All six samples are used to measure MSE after each epoch.
inputs = torch.tensor([[x1, x2] for x1, x2, y in data], dtype=torch.float32)
targets = torch.tensor([[y] for x1, x2, y in data], dtype=torch.float32)
loss_history = []
with torch.no_grad():
    initial_mse = criterion(net(inputs), targets).item()
print(f"Initial MSE: {initial_mse:.6f}")

net.train()
for epoch in range(100):
```

```
    for x1, x2, y in data:
        X = torch.tensor([x1, x2], dtype=torch.float32)
        Y = torch.tensor([y], dtype=torch.float32)
        optimizer.zero_grad()  # Clear gradients from the previous sample.
        outputs = net(X)
        loss = criterion(outputs, Y)
        loss.backward()        # Compute gradients for both weights.
        optimizer.step()       # Update weights using SGD with momentum.

    with torch.no_grad():
        epoch_mse = criterion(net(inputs), targets).item()
    loss_history.append(epoch_mse)
    if epoch == 0 or (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch + 1:3d} - MSE: {epoch_mse:.8e}")

# Test the trained network on the supplied training samples.
net.eval()
with torch.no_grad():
    predictions = net(inputs)
    final_mse = criterion(predictions, targets).item()
    w1, w2 = net.fc1.weight[0].tolist()

print(f"\nLearned model: y = {w1:.6f} * x1 + {w2:.6f} * x2")
print(f"Final training MSE: {final_mse:.3e}")
print("\nx1     x2      target   prediction")
for (x1, x2, y), prediction in zip(data, predictions):
    print(f"{x1:4.1f}   {x2:5.1f}   {y:7.1f}   {prediction.item():10.6f}")
```

```
Net(
  (fc1): Linear(in_features=2, out_features=1, bias=False)
)
[Parameter containing:
tensor([[-0.0053,  0.3793]], requires_grad=True)]

Initial MSE: 94.615715
Epoch   1 - MSE: 3.71837120e+01
Epoch  10 - MSE: 5.88038862e-02
Epoch  20 - MSE: 2.42577426e-05
Epoch  30 - MSE: 1.29758371e-09
Epoch  40 - MSE: 9.09494702e-13
Epoch  50 - MSE: 5.84539838e-12
Epoch  60 - MSE: 5.84539838e-12
Epoch  70 - MSE: 5.76013326e-12
Epoch  80 - MSE: 5.76013326e-12
Epoch  90 - MSE: 5.76013326e-12
Epoch 100 - MSE: 5.84539838e-12

Learned model: y = 3.000000 * x1 + -0.000001 * x2
Final training MSE: 5.845e-12

x1     x2      target   prediction
 1.0     2.1       3.0     2.999999
 2.0     3.5       6.0     5.999998
 3.0     3.0       9.0     8.999999
 4.0     2.1      12.0    12.000000
 5.0     7.2      15.0    14.999996
 6.0    10.1      18.0    17.999996
```

Question 2: Manual Backpropagation

One iteration and verification with PyTorch

Scope. Calculate the gradients at the notebook's initial weights, using one input–target pair and one backward pass, before any weight update. Vectors are row vectors; $(W_\ell)_{ij}$ connects input i to output j. Both layers have no bias.

1. Initial values and forward pass

The notebook specifies

$$X = \begin{bmatrix} 1 & 2 \end{bmatrix}, \quad T = \begin{bmatrix} 0 & 0 \end{bmatrix}, \quad W_1 = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix}.$$

The hidden-layer preactivation and activation are

$$\begin{aligned} S_2 = XW_1 &= \begin{bmatrix} 1(2) + 2(0) & 1(1) + 2(1) \end{bmatrix} = \begin{bmatrix} 2 & 3 \end{bmatrix}, \\ Y_2 = \mathrm{ReLU}(S_2) &= \begin{bmatrix} 2 & 3 \end{bmatrix}. \end{aligned}$$

The output-layer preactivation and activation are

$$\begin{aligned} S_3 = Y_2W_2 &= \begin{bmatrix} 2(1) + 3(1) & 2(2) + 3(2) \end{bmatrix} = \begin{bmatrix} 5 & 10 \end{bmatrix}, \\ Y_3 = \mathrm{ReLU}(S_3) &= \begin{bmatrix} 5 & 10 \end{bmatrix}. \end{aligned}$$

All four preactivations are positive. Hence every ReLU derivative used in this iteration is 1.

2. Mean squared error and output error signal

The default `nn.MSELoss()` averages over the two output elements [2]. Therefore

$$L = \frac{(y_{3,1} - t_1)^2 + (y_{3,2} - t_2)^2}{2} = \frac{5^2 + 10^2}{2} = 62.5.$$

For each output j,

$$\frac{\partial L}{\partial y_{3,j}} = \frac{2}{2}(y_{3,j} - t_j), \qquad \delta_3 = \frac{\partial L}{\partial S_3} = \begin{bmatrix} 5 & 10 \end{bmatrix} \odot \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} 5 & 10 \end{bmatrix}.$$

Here $\odot$ denotes elementwise multiplication.

3. Gradient of the second-layer weights

Since $s_{3,j} = \sum_i y_{2,i}(W_2)_{ij}$, the chain rule gives

$$\frac{\partial L}{\partial (W_2)_{ij}} = y_{2,i}\delta_{3,j}.$$

Thus all four derivatives are the entries of the outer product

$$\boxed{\nabla_{W_2} L = Y_2^{\mathsf{T}} \delta_3 = \begin{bmatrix} 2 \cdot 5 & 2 \cdot 10 \\ 3 \cdot 5 & 3 \cdot 10 \end{bmatrix} = \begin{bmatrix} 10 & 20 \\ 15 & 30 \end{bmatrix}.}$$

4. Hidden error signal and gradient of the first-layer weights

Each hidden activation affects both outputs. Applying the chain rule,

$$\frac{\partial L}{\partial y_{2,i}} = \sum_j \delta_{3,j}(W_2)_{ij}, \qquad \frac{\partial L}{\partial Y_2} = \delta_3 W_2^\mathsf{T}.$$

In numbers,

$$\delta_3 W_2^\mathsf{T} = \begin{bmatrix}5 & 10\end{bmatrix}\begin{bmatrix}1 & 1\\ 2 & 2\end{bmatrix} = \begin{bmatrix}5(1)+10(2) & 5(1)+10(2)\end{bmatrix} = \begin{bmatrix}25 & 25\end{bmatrix}.$$

Both hidden ReLUs have derivative 1, so

$$\delta_2 = \frac{\partial L}{\partial S_2} = (\delta_3 W_2^\mathsf{T}) \odot \mathrm{ReLU}'(S_2) = \begin{bmatrix}25 & 25\end{bmatrix}.$$

Since $s_{2,j} = \sum_i x_i (W_1)_{ij}$,

$$\frac{\partial L}{\partial (W_1)_{ij}} = x_i \delta_{2,j}, \qquad \boxed{\nabla_{W_1} L = X^\mathsf{T}\delta_2 = \begin{bmatrix}1\cdot 25 & 1\cdot 25\\ 2\cdot 25 & 2\cdot 25\end{bmatrix} = \begin{bmatrix}25 & 25\\ 50 & 50\end{bmatrix}}.$$

5. Verification and the `nn.Linear` transpose

The notebook's direct `torch.matmul` implementation prints

$$\texttt{W1.grad} = \begin{bmatrix}25 & 25\\ 50 & 50\end{bmatrix}, \qquad \texttt{W2.grad} = \begin{bmatrix}10 & 20\\ 15 & 30\end{bmatrix}.$$

These exactly match the manual derivatives above.

PyTorch's `nn.Linear` computes $xA^\mathsf{T} + b$ and stores A with shape (output features, input features) [1]. The notebook sets

$$A_1 = W_1^\mathsf{T} = \begin{bmatrix}2 & 0\\ 1 & 1\end{bmatrix}, \qquad A_2 = W_2^\mathsf{T} = \begin{bmatrix}1 & 1\\ 2 & 2\end{bmatrix}.$$

Thus $XA_1^\mathsf{T} = XW_1$ and $Y_2 A_2^\mathsf{T} = Y_2 W_2$. Since $(A_\ell)_{ji} = (W_\ell)_{ij}$, their parameter gradients obey

$$\nabla_{A_\ell} L = (\nabla_{W_\ell} L)^\mathsf{T}.$$

The executed notebook confirms

$$\texttt{fc1.weight.grad} = \begin{bmatrix}25 & 50\\ 25 & 50\end{bmatrix}, \qquad \texttt{fc2.weight.grad} = \begin{bmatrix}10 & 15\\ 20 & 30\end{bmatrix}.$$

Comparing the notebook's first backward-pass outputs with the manual matrices gives maximum absolute error 0 for both direct gradients and both transposed `nn.Linear` gradients. Both implementations give $Y_3 = [5, 10]$ and $L = 62.5$ before the first update.

References

[1] PyTorch documentation, `torch.nn.Linear`. `https://docs.pytorch.org/docs/2.14/generated/torch.nn.Linear.html`

[2] PyTorch documentation, `torch.nn.MSELoss`. `https://docs.pytorch.org/docs/2.14/generated/torch.nn.MSELoss.html`

Q4: MNIST Multi-Layer Perceptron

Parameter count and operations per input image

Architecture. In the supplied Code4 GPU notebook, a 28×28 grayscale image is flattened to 784 inputs. The three fully connected layers are

$$784 \xrightarrow{\texttt{fc1}} 512 \xrightarrow{\texttt{fc2}} 512 \xrightarrow{\texttt{fc3}} 10.$$

Each layer has a bias for every output neuron. The actual forward method applies ReLU after *all three layers*, including the final 10 outputs.

1. Total parameters

A fully connected layer with n_{in} inputs and n_{out} outputs has $n_{\text{in}}n_{\text{out}}$ weights and n_{out} independent bias parameters:

$$P = n_{\text{in}}n_{\text{out}} + n_{\text{out}}.$$

Layer	Weights	Bias parameters	Total parameters
`fc1`: $784 \to 512$	$784(512) = 401{,}408$	512	401,920
`fc2`: $512 \to 512$	$512(512) = 262{,}144$	512	262,656
`fc3`: $512 \to 10$	$512(10) = 5{,}120$	10	5,130
Total	668,672	1,034	669,706

$$\boxed{P_{\text{total}} = 401{,}920 + 262{,}656 + 5{,}130 = 669{,}706 \text{ parameters.}}$$

Flattening and ReLU introduce no trainable parameters.

2. Total operations per image

Each output neuron computes a dot product with n_{in} inputs, giving $n_{\text{in}}n_{\text{out}}$ MACs per layer. Using the specified rules, one MAC costs 2 operations, one bias addition costs 1, and one ReLU costs 1. Therefore, for each layer in this network,

$$O = \underbrace{2n_{\text{in}}n_{\text{out}}}_{\text{MAC operations}} + \underbrace{n_{\text{out}}}_{\text{bias additions}} + \underbrace{n_{\text{out}}}_{\text{ReLU operations}}.$$

Layer	MACs	MAC operations	Bias adds	ReLUs	Total operations
`fc1`	401,408	802,816	512	512	803,840
`fc2`	262,144	524,288	512	512	525,312
`fc3`	5,120	10,240	10	10	10,260
Total	668,672	1,337,344	1,034	1,034	1,339,412

$$\boxed{O_{\text{total}} = 803{,}840 + 525{,}312 + 10{,}260 = 1{,}339{,}412 \text{ operations per image.}}$$

Counting scope. The total covers one forward pass on a preprocessed image. Flattening adds no arithmetic. Training, preprocessing, and device transfers are excluded; GPU execution does not change this count. The final `argmax` has no specified comparison cost. If one comparison costs one operation, selecting the largest of 10 outputs adds $10 - 1 = 9$, giving 1,339,421 operations including class selection.

Verification. Instantiating the notebook's `Net` class and summing `p.numel()` over its parameters gives 669,706. Layer dimensions and the three ReLU calls give the operation totals above.

Question 5: Nonlinear Perceptron Training

[1]:
```
### Training with fancier version ###

import torch
import torch.nn as nn
import torch.nn.functional as F

import torch.optim as optim
import matplotlib.pyplot as plt

class Net(nn.Module): ## nn.Module class is used
    def __init__(self):
        torch.manual_seed(0)
        super(Net, self).__init__()
        self.fc1 = nn.Linear(1,16)
        self.fc2 = nn.Linear(16,16)
        self.fc3 = nn.Linear(16,1)

    def forward(self, x):
        x = (x - 3.5) / 2.5
        x = torch.tanh(self.fc1(x))
        x = torch.tanh(self.fc2(x))
        x = self.fc3(x)
        return x

net = Net()

print(net)
print(list(net.parameters())) # parameters are randomized


#def criterion(out, label):
#    return (label - out)**2
criterion = nn.MSELoss()


optimizer = optim.Adam(net.parameters(), lr=0.05)
#optimizer = optim.Adam(net.parameters(), lr=0.1)

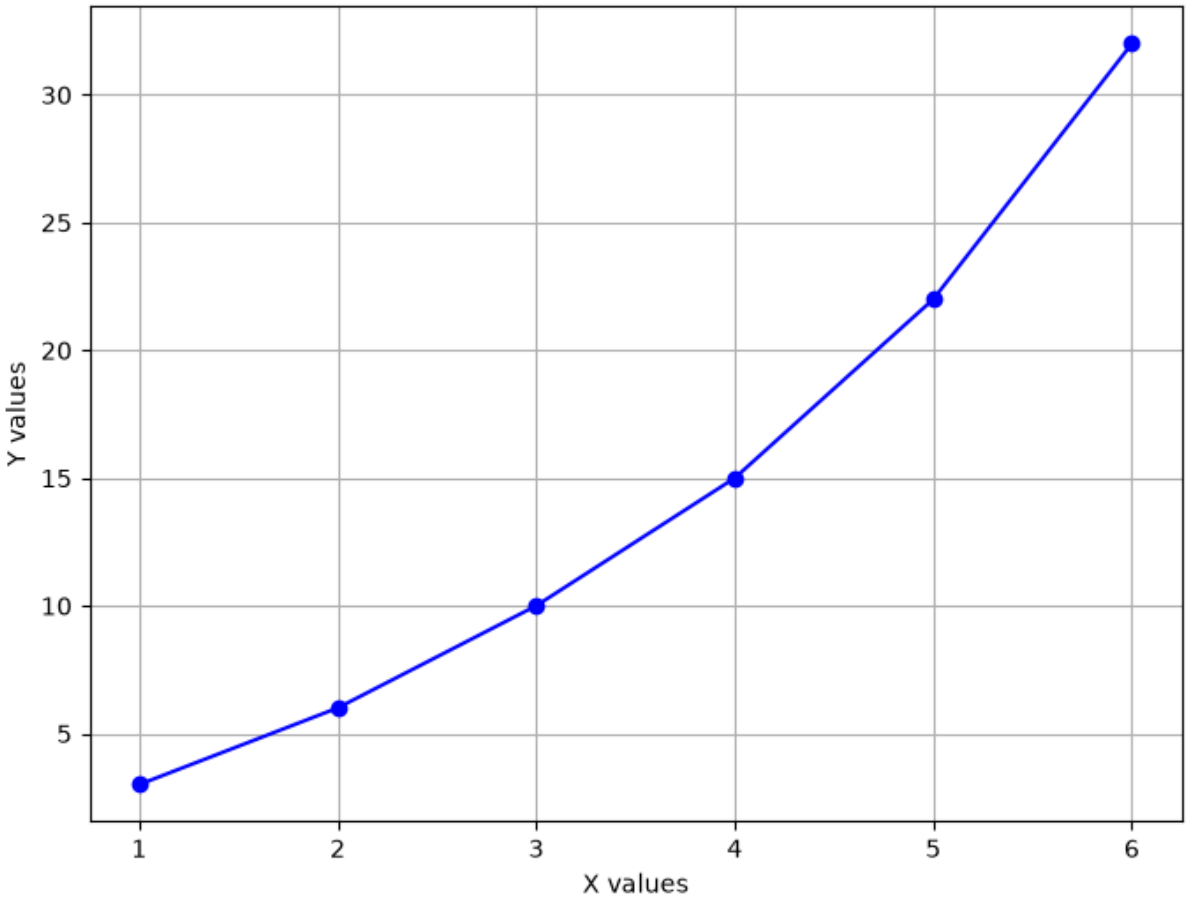

data = [(1.0,3.0), (2.0,6.0), (3.0,10.0), (4.0,15.0), (5.0,22.0), (6.0,32.0)]

# Split data into x and y
x_values, y_values = zip(*data)

# Plotting
plt.figure(figsize=(8, 6))
plt.plot(x_values, y_values, marker='o', linestyle='-', color='b')
plt.xlabel('X values')
plt.ylabel('Y values')
plt.grid(True)
plt.show()
```

```
Net(
  (fc1): Linear(in_features=1, out_features=16, bias=True)
  (fc2): Linear(in_features=16, out_features=16, bias=True)
  (fc3): Linear(in_features=16, out_features=1, bias=True)
)
[Parameter containing:
tensor([[-0.0075],
        [ 0.5364],
        [-0.8230],
        [-0.7359],
        [-0.3852],
        [ 0.2682],
        [-0.0198],
        [ 0.7929],
        [-0.0887],
        [ 0.2646],
        [-0.3022],
        [-0.1966],
        [-0.9553],
        [-0.6623],
        [-0.4122],
        [ 0.0370]], requires_grad=True), Parameter containing:
tensor([ 0.3953,  0.6000, -0.6779, -0.4355,  0.3632,  0.8304, -0.2058,  0.7483,
        -0.1612,  0.1058,  0.9055, -0.9277, -0.6295, -0.2532, -0.3898,  0.8640],
       requires_grad=True), Parameter containing:
tensor([[-0.1620, -0.1151, -0.1747, -0.2341, -0.1459,  0.2149,  0.1116,  0.1212,
          0.0131, -0.1282,  0.0423, -0.2334, -0.1806, -0.1289,  0.1577,
0.1466],
        [-0.1109, -0.0090,  0.1599,  0.2485,  0.0992,  0.0338,  0.1676, -0.1472,
          0.0466, -0.1938, -0.1733, -0.1291,  0.1131,  0.1005, -0.1481,
0.0755],
        [ 0.1372, -0.0316,  0.0095,  0.0579,  0.1551,  0.2400, -0.1927, -0.0916,
          0.0983,  0.2071,  0.2176,  0.2206,  0.0498, -0.2174,  0.0230,
-0.1564],
        [-0.2330,  0.2221,  0.1901, -0.2494,  0.0468, -0.0421, -0.0411, -0.1144,
          0.0961, -0.1481,  0.0916,  0.1264,  0.1790,  0.0935, -0.2474,
-0.1622],
        [ 0.1248,  0.0523, -0.1950, -0.1440,  0.2352,  0.1685, -0.1090, -0.0629,
         -0.2381, -0.0045, -0.1883, -0.1928, -0.0138,  0.0375, -0.1024,
0.1483],
        [-0.1521,  0.2268,  0.1713, -0.2108, -0.0622,  0.0113,  0.0365,  0.0593,
          0.0981,  0.0150, -0.1220,  0.1183, -0.2398, -0.1482, -0.0626,
-0.1218],
        [-0.0875, -0.2049, -0.0532,  0.0534, -0.1629, -0.0128,  0.1790, -0.0257,
          0.0069, -0.0216,  0.0506,  0.1590,  0.2368,  0.1588,  0.2374,
-0.0181],
        [-0.2246, -0.1185,  0.1702, -0.0016, -0.1243, -0.1916, -0.2340, -0.2110,
         -0.0507,  0.1371,  0.1352, -0.2411,  0.1559, -0.1956, -0.0529,
-0.1014],
        [-0.0482, -0.0491, -0.2243, -0.2159, -0.0391,  0.0032, -0.1136,  0.0942,
         -0.2250, -0.0169,  0.2199, -0.1020,  0.2258,  0.0905, -0.2256,
0.1582],
        [-0.0288, -0.1116,  0.1999, -0.2020,  0.0268, -0.0523,  0.1785,  0.0698,
          0.1201,  0.0883, -0.0601, -0.0526, -0.2060,  0.1355,  0.1985,
0.1711],
        [-0.1763,  0.0111, -0.1762, -0.1376, -0.1457,  0.0854, -0.1490, -0.0055,
          0.0105,  0.1612, -0.1890, -0.1716, -0.1452,  0.1750, -0.0899,
0.2109],
        [ 0.0904,  0.0317, -0.0019, -0.0494,  0.0314, -0.0571, -0.0018,  0.0319,
         -0.1956, -0.1310,  0.2019, -0.2029, -0.0180,  0.2473,  0.0903,
0.0071],
        [-0.2167,  0.1238, -0.1781, -0.0710, -0.0839, -0.0370,  0.0027,  0.2062,
          0.0312,  0.2239,  0.1529, -0.1581,  0.1121, -0.1767, -0.1060,
```

```
0.0735],
        [ 0.0825,  0.1876, -0.0805,  0.0004,  0.1287, -0.2418,  0.1807, -0.2067,
          0.0034, -0.0425, -0.1317,  0.0330,  0.2067, -0.0731, -0.1484,
-0.0925],
        [-0.2478,  0.1128, -0.1201, -0.1668, -0.1440,  0.1437,  0.1324,  0.1919,
          0.0907, -0.0835, -0.0699,  0.0739,  0.2055,  0.0680, -0.1183,
-0.1175],
        [-0.2364,  0.0540, -0.1403, -0.2229,  0.2192, -0.1624, -0.0284,  0.0716,
          0.0080, -0.1682, -0.2021,  0.1993,  0.0407,  0.2074, -0.0838,
0.0736]],
       requires_grad=True), Parameter containing:
tensor([-0.0572, -0.0111, -0.1523,  0.0846,  0.0790, -0.0052, -0.0562, -0.1541,
         0.1729, -0.1861,  0.1024, -0.0841, -0.1206,  0.0449, -0.1299,  0.0576],
       requires_grad=True), Parameter containing:
tensor([[ 0.0491, -0.1856,  0.0416,  0.1065,  0.0990, -0.0315, -0.2049, -0.0385,
          0.0868, -0.0912,  0.0949,  0.1665, -0.1305,  0.0025,  0.1034,
0.0196]],
       requires_grad=True), Parameter containing:
tensor([0.0209], requires_grad=True)]
```



[2]:
```
inputs = torch.FloatTensor([[x] for x, y in data])
labels = torch.FloatTensor([[y] for x, y in data])

for epoch in range(10000):
    optimizer.zero_grad()
    outputs = net(inputs)
```

```
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()
    with torch.no_grad():
        loss = criterion(net(inputs), labels)
    if epoch == 0 or (epoch + 1) % 50 == 0 or loss.item() < 1e-5:
        print("Epoch {} - loss: {}".format(epoch + 1, loss.item()))
    if loss.item() < 1e-5:
        break
else:
    raise RuntimeError("Training did not reach MSE below 1e-5")


### Test the trained network ###
for i, current_data in enumerate(data):
    X, Y = current_data
    X, Y = torch.FloatTensor([X]), torch.FloatTensor([Y])
    out = net(torch.FloatTensor(X))
    print("when x = {}, y = {}".format(X, out))
```

```
Epoch 1 - loss: 279.7319030761719
Epoch 50 - loss: 9.934516906738281
Epoch 100 - loss: 0.024694988504052162
Epoch 150 - loss: 0.0002745965321082622
Epoch 163 - loss: 4.8735860218585e-06
when x = tensor([1.]), y = tensor([2.9972], grad_fn=<ViewBackward0>)
when x = tensor([2.]), y = tensor([5.9974], grad_fn=<ViewBackward0>)
when x = tensor([3.]), y = tensor([10.0009], grad_fn=<ViewBackward0>)
when x = tensor([4.]), y = tensor([15.0014], grad_fn=<ViewBackward0>)
when x = tensor([5.]), y = tensor([22.0030], grad_fn=<ViewBackward0>)
when x = tensor([6.]), y = tensor([32.0018], grad_fn=<ViewBackward0>)
```

[3]:
```
X
```

[3]:
```
tensor([6.])
```

[4]:
```
for epoch in range(20):
    print(epoch)
```

```
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
```

[5]:
```
W = torch.tensor([1.0], requires_grad=True)
W = W*2
label = 1.0
loss = W*5 - label
loss.backward()
W.grad
```

```
/var/folders/yj/gx_cp2255q7_5bglq236my680000gn/T/ipykernel_2609/353321738.py:6:
UserWarning: The .grad attribute of a Tensor that is not a leaf Tensor is being
accessed. Its .grad attribute won't be populated during autograd.backward(). If
you indeed want the .grad field to be populated for a non-leaf Tensor, use
.retain_grad() on the non-leaf Tensor. If you access the non-leaf Tensor by
mistake, make sure you access the leaf Tensor instead. See
github.com/pytorch/pytorch/pull/30531 for more information. (Triggered
internally at
/Users/runner/work/pytorch/pytorch/build/aten/src/ATen/core/TensorBody.h:493.)
  W.grad
```

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

Question 6: Batch versus Per-Point SGD

[1]:
```
import torch
from torch.autograd import Variable
import torch.nn as nn
import torch.nn.functional as F

import torch.optim as optim
import matplotlib.pyplot as plt

class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(2,1,bias=False)
    def forward(self, x):
        x = self.fc1(x)
        return x

net = Net()
print(net)


net.fc1.weight = torch.nn.Parameter(torch.tensor([[1., -1.]], requires_grad=True))

print(list(net.parameters()))

#input = torch.randn(1,2)
#out = net(input)


criterion = nn.MSELoss()
#def criterion(out, label):
#    return ((label - out)**2).mean()


optimizer = optim.SGD(net.parameters(), lr=0.01, momentum=0.5)
#optimizer = optim.Adam(net.parameters(), lr=0.005)


data = torch.tensor([[1.,3.], [2.,6.], [3.,9.]], dtype=torch.float)
target = torch.tensor([[1.],[5.],[13.]], dtype=torch.float)

hist = []

############### Batch GD based update ###############

for epoch in range(300):
    optimizer.zero_grad()
    outputs = net(data)
    loss = criterion(outputs, target)
    loss.backward()
    hist.append(loss.detach())
    optimizer.step()
```

```
    print("Epoch {} - loss: {}".format(epoch, loss))
#####################################################

### Test the trained network ###
for i, current_data in enumerate(data):
    out = net(current_data)
    print("when x = {}, y = {}".format(current_data, out))

plt.plot(hist, label = "training curve")
```

```
Net(
  (fc1): Linear(in_features=2, out_features=1, bias=False)
)
[Parameter containing:
tensor([[ 1., -1.]], requires_grad=True)]

Epoch 0 - loss: 150.3333282470703
Epoch 1 - loss: 6.120000839233398
Epoch 2 - loss: 36.42483139038086
Epoch 3 - loss: 18.104551315307617
Epoch 4 - loss: 6.065778732299805
Epoch 5 - loss: 10.36883544921875
Epoch 6 - loss: 6.2322306632995605
Epoch 7 - loss: 5.852260589599609
Epoch 8 - loss: 6.088119983673096
Epoch 9 - loss: 5.494865417480469
Epoch 10 - loss: 5.574592590332031
Epoch 11 - loss: 5.536750793457031
Epoch 12 - loss: 5.476492404937744
Epoch 13 - loss: 5.4938530921936035
Epoch 14 - loss: 5.480627536773682
Epoch 15 - loss: 5.477013111114502
Epoch 16 - loss: 5.478647232055664
Epoch 17 - loss: 5.476379871368408
Epoch 18 - loss: 5.476480007171631
Epoch 19 - loss: 5.4764628410339355
Epoch 20 - loss: 5.47619104385376
Epoch 21 - loss: 5.476251602172852
Epoch 22 - loss: 5.476212978363037
Epoch 23 - loss: 5.476190567016602
Epoch 24 - loss: 5.476199626922607
Epoch 25 - loss: 5.476192474365234
Epoch 26 - loss: 5.476190567016602
Epoch 27 - loss: 5.476191997528076
Epoch 28 - loss: 5.476190090179443
Epoch 29 - loss: 5.476190567016602
Epoch 30 - loss: 5.476190090179443
Epoch 31 - loss: 5.476190567016602
Epoch 32 - loss: 5.47619104385376
Epoch 33 - loss: 5.476190090179443
Epoch 34 - loss: 5.476190090179443
Epoch 35 - loss: 5.476190567016602
Epoch 36 - loss: 5.476190090179443
Epoch 37 - loss: 5.47619104385376
Epoch 38 - loss: 5.47619104385376
Epoch 39 - loss: 5.476190567016602
Epoch 40 - loss: 5.47619104385376
Epoch 41 - loss: 5.476188659667969
Epoch 42 - loss: 5.476190090179443
Epoch 43 - loss: 5.476189136505127
Epoch 44 - loss: 5.476190090179443
Epoch 45 - loss: 5.476191997528076
Epoch 46 - loss: 5.476191997528076
Epoch 47 - loss: 5.476190090179443
```

```
Epoch 48 - loss: 5.476190567016602
Epoch 49 - loss: 5.476190567016602
Epoch 50 - loss: 5.476190090179443
Epoch 51 - loss: 5.47619104385376
Epoch 52 - loss: 5.476190567016602
Epoch 53 - loss: 5.47619104385376
Epoch 54 - loss: 5.47619104385376
Epoch 55 - loss: 5.47619104385376
Epoch 56 - loss: 5.476190567016602
Epoch 57 - loss: 5.47619104385376
Epoch 58 - loss: 5.47619104385376
Epoch 59 - loss: 5.47619104385376
Epoch 60 - loss: 5.476190567016602
Epoch 61 - loss: 5.47619104385376
Epoch 62 - loss: 5.47619104385376
Epoch 63 - loss: 5.47619104385376
Epoch 64 - loss: 5.476190567016602
Epoch 65 - loss: 5.47619104385376
Epoch 66 - loss: 5.47619104385376
Epoch 67 - loss: 5.47619104385376
Epoch 68 - loss: 5.476190567016602
Epoch 69 - loss: 5.47619104385376
Epoch 70 - loss: 5.47619104385376
Epoch 71 - loss: 5.47619104385376
Epoch 72 - loss: 5.476190567016602
Epoch 73 - loss: 5.47619104385376
Epoch 74 - loss: 5.47619104385376
Epoch 75 - loss: 5.47619104385376
Epoch 76 - loss: 5.476190567016602
Epoch 77 - loss: 5.47619104385376
Epoch 78 - loss: 5.47619104385376
Epoch 79 - loss: 5.47619104385376
Epoch 80 - loss: 5.476190567016602
Epoch 81 - loss: 5.47619104385376
Epoch 82 - loss: 5.47619104385376
Epoch 83 - loss: 5.47619104385376
Epoch 84 - loss: 5.476190567016602
Epoch 85 - loss: 5.47619104385376
Epoch 86 - loss: 5.47619104385376
Epoch 87 - loss: 5.47619104385376
Epoch 88 - loss: 5.476190567016602
Epoch 89 - loss: 5.47619104385376
Epoch 90 - loss: 5.47619104385376
Epoch 91 - loss: 5.47619104385376
Epoch 92 - loss: 5.476190567016602
Epoch 93 - loss: 5.47619104385376
Epoch 94 - loss: 5.47619104385376
Epoch 95 - loss: 5.47619104385376
Epoch 96 - loss: 5.476190567016602
Epoch 97 - loss: 5.47619104385376
Epoch 98 - loss: 5.47619104385376
Epoch 99 - loss: 5.47619104385376
Epoch 100 - loss: 5.476190567016602
Epoch 101 - loss: 5.47619104385376
Epoch 102 - loss: 5.47619104385376
Epoch 103 - loss: 5.47619104385376
Epoch 104 - loss: 5.476190567016602
Epoch 105 - loss: 5.47619104385376
Epoch 106 - loss: 5.47619104385376
Epoch 107 - loss: 5.47619104385376
Epoch 108 - loss: 5.476190567016602
Epoch 109 - loss: 5.47619104385376
Epoch 110 - loss: 5.47619104385376
```

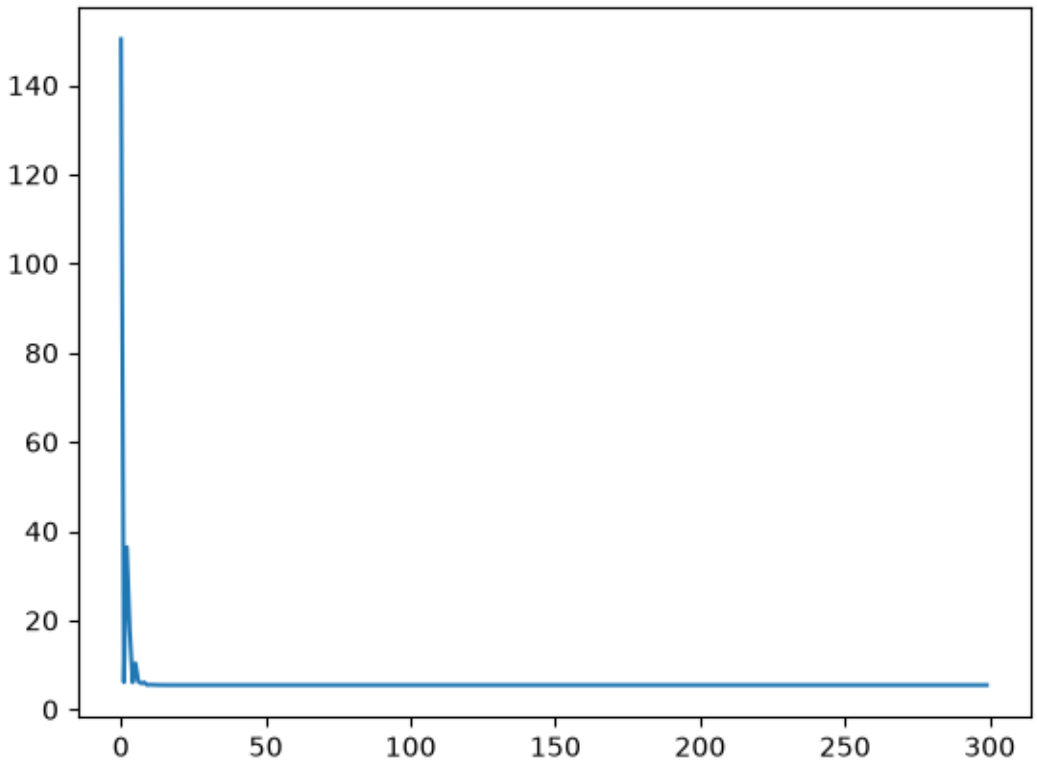
```
Epoch 111 - loss: 5.47619104385376
Epoch 112 - loss: 5.476190567016602
Epoch 113 - loss: 5.47619104385376
Epoch 114 - loss: 5.47619104385376
Epoch 115 - loss: 5.47619104385376
Epoch 116 - loss: 5.476190567016602
Epoch 117 - loss: 5.47619104385376
Epoch 118 - loss: 5.47619104385376
Epoch 119 - loss: 5.47619104385376
Epoch 120 - loss: 5.476190567016602
Epoch 121 - loss: 5.47619104385376
Epoch 122 - loss: 5.47619104385376
Epoch 123 - loss: 5.47619104385376
Epoch 124 - loss: 5.476190567016602
Epoch 125 - loss: 5.47619104385376
Epoch 126 - loss: 5.47619104385376
Epoch 127 - loss: 5.47619104385376
Epoch 128 - loss: 5.476190567016602
Epoch 129 - loss: 5.47619104385376
Epoch 130 - loss: 5.47619104385376
Epoch 131 - loss: 5.47619104385376
Epoch 132 - loss: 5.476190567016602
Epoch 133 - loss: 5.47619104385376
Epoch 134 - loss: 5.47619104385376
Epoch 135 - loss: 5.47619104385376
Epoch 136 - loss: 5.476190567016602
Epoch 137 - loss: 5.47619104385376
Epoch 138 - loss: 5.47619104385376
Epoch 139 - loss: 5.47619104385376
Epoch 140 - loss: 5.476190567016602
Epoch 141 - loss: 5.47619104385376
Epoch 142 - loss: 5.47619104385376
Epoch 143 - loss: 5.47619104385376
Epoch 144 - loss: 5.476190567016602
Epoch 145 - loss: 5.47619104385376
Epoch 146 - loss: 5.47619104385376
Epoch 147 - loss: 5.47619104385376
Epoch 148 - loss: 5.476190567016602
Epoch 149 - loss: 5.47619104385376
Epoch 150 - loss: 5.47619104385376
Epoch 151 - loss: 5.47619104385376
Epoch 152 - loss: 5.476190567016602
Epoch 153 - loss: 5.47619104385376
Epoch 154 - loss: 5.47619104385376
Epoch 155 - loss: 5.47619104385376
Epoch 156 - loss: 5.476190567016602
Epoch 157 - loss: 5.47619104385376
Epoch 158 - loss: 5.47619104385376
Epoch 159 - loss: 5.47619104385376
Epoch 160 - loss: 5.476190567016602
Epoch 161 - loss: 5.47619104385376
Epoch 162 - loss: 5.47619104385376
Epoch 163 - loss: 5.47619104385376
Epoch 164 - loss: 5.476190567016602
Epoch 165 - loss: 5.47619104385376
Epoch 166 - loss: 5.47619104385376
Epoch 167 - loss: 5.47619104385376
Epoch 168 - loss: 5.476190567016602
Epoch 169 - loss: 5.47619104385376
Epoch 170 - loss: 5.47619104385376
Epoch 171 - loss: 5.47619104385376
Epoch 172 - loss: 5.476190567016602
Epoch 173 - loss: 5.47619104385376
```

```
Epoch 174 - loss: 5.47619104385376
Epoch 175 - loss: 5.47619104385376
Epoch 176 - loss: 5.476190567016602
Epoch 177 - loss: 5.47619104385376
Epoch 178 - loss: 5.47619104385376
Epoch 179 - loss: 5.47619104385376
Epoch 180 - loss: 5.476190567016602
Epoch 181 - loss: 5.47619104385376
Epoch 182 - loss: 5.47619104385376
Epoch 183 - loss: 5.47619104385376
Epoch 184 - loss: 5.476190567016602
Epoch 185 - loss: 5.47619104385376
Epoch 186 - loss: 5.47619104385376
Epoch 187 - loss: 5.47619104385376
Epoch 188 - loss: 5.476190567016602
Epoch 189 - loss: 5.47619104385376
Epoch 190 - loss: 5.47619104385376
Epoch 191 - loss: 5.47619104385376
Epoch 192 - loss: 5.476190567016602
Epoch 193 - loss: 5.47619104385376
Epoch 194 - loss: 5.47619104385376
Epoch 195 - loss: 5.47619104385376
Epoch 196 - loss: 5.476190567016602
Epoch 197 - loss: 5.47619104385376
Epoch 198 - loss: 5.47619104385376
Epoch 199 - loss: 5.47619104385376
Epoch 200 - loss: 5.476190567016602
Epoch 201 - loss: 5.47619104385376
Epoch 202 - loss: 5.47619104385376
Epoch 203 - loss: 5.47619104385376
Epoch 204 - loss: 5.476190567016602
Epoch 205 - loss: 5.47619104385376
Epoch 206 - loss: 5.47619104385376
Epoch 207 - loss: 5.47619104385376
Epoch 208 - loss: 5.476190567016602
Epoch 209 - loss: 5.47619104385376
Epoch 210 - loss: 5.47619104385376
Epoch 211 - loss: 5.47619104385376
Epoch 212 - loss: 5.476190567016602
Epoch 213 - loss: 5.47619104385376
Epoch 214 - loss: 5.47619104385376
Epoch 215 - loss: 5.47619104385376
Epoch 216 - loss: 5.476190567016602
Epoch 217 - loss: 5.47619104385376
Epoch 218 - loss: 5.47619104385376
Epoch 219 - loss: 5.47619104385376
Epoch 220 - loss: 5.476190567016602
Epoch 221 - loss: 5.47619104385376
Epoch 222 - loss: 5.47619104385376
Epoch 223 - loss: 5.47619104385376
Epoch 224 - loss: 5.476190567016602
Epoch 225 - loss: 5.47619104385376
Epoch 226 - loss: 5.47619104385376
Epoch 227 - loss: 5.47619104385376
Epoch 228 - loss: 5.476190567016602
Epoch 229 - loss: 5.47619104385376
Epoch 230 - loss: 5.47619104385376
Epoch 231 - loss: 5.47619104385376
Epoch 232 - loss: 5.476190567016602
Epoch 233 - loss: 5.47619104385376
Epoch 234 - loss: 5.47619104385376
Epoch 235 - loss: 5.47619104385376
Epoch 236 - loss: 5.476190567016602
```

```
Epoch 237 - loss: 5.47619104385376
Epoch 238 - loss: 5.47619104385376
Epoch 239 - loss: 5.47619104385376
Epoch 240 - loss: 5.476190567016602
Epoch 241 - loss: 5.47619104385376
Epoch 242 - loss: 5.47619104385376
Epoch 243 - loss: 5.47619104385376
Epoch 244 - loss: 5.476190567016602
Epoch 245 - loss: 5.47619104385376
Epoch 246 - loss: 5.47619104385376
Epoch 247 - loss: 5.47619104385376
Epoch 248 - loss: 5.476190567016602
Epoch 249 - loss: 5.47619104385376
Epoch 250 - loss: 5.47619104385376
Epoch 251 - loss: 5.47619104385376
Epoch 252 - loss: 5.476190567016602
Epoch 253 - loss: 5.47619104385376
Epoch 254 - loss: 5.47619104385376
Epoch 255 - loss: 5.47619104385376
Epoch 256 - loss: 5.476190567016602
Epoch 257 - loss: 5.47619104385376
Epoch 258 - loss: 5.47619104385376
Epoch 259 - loss: 5.47619104385376
Epoch 260 - loss: 5.476190567016602
Epoch 261 - loss: 5.47619104385376
Epoch 262 - loss: 5.47619104385376
Epoch 263 - loss: 5.47619104385376
Epoch 264 - loss: 5.476190567016602
Epoch 265 - loss: 5.47619104385376
Epoch 266 - loss: 5.47619104385376
Epoch 267 - loss: 5.47619104385376
Epoch 268 - loss: 5.476190567016602
Epoch 269 - loss: 5.47619104385376
Epoch 270 - loss: 5.47619104385376
Epoch 271 - loss: 5.47619104385376
Epoch 272 - loss: 5.476190567016602
Epoch 273 - loss: 5.47619104385376
Epoch 274 - loss: 5.47619104385376
Epoch 275 - loss: 5.47619104385376
Epoch 276 - loss: 5.476190567016602
Epoch 277 - loss: 5.47619104385376
Epoch 278 - loss: 5.47619104385376
Epoch 279 - loss: 5.47619104385376
Epoch 280 - loss: 5.476190567016602
Epoch 281 - loss: 5.47619104385376
Epoch 282 - loss: 5.47619104385376
Epoch 283 - loss: 5.47619104385376
Epoch 284 - loss: 5.476190567016602
Epoch 285 - loss: 5.47619104385376
Epoch 286 - loss: 5.47619104385376
Epoch 287 - loss: 5.47619104385376
Epoch 288 - loss: 5.476190567016602
Epoch 289 - loss: 5.47619104385376
Epoch 290 - loss: 5.47619104385376
Epoch 291 - loss: 5.47619104385376
Epoch 292 - loss: 5.476190567016602
Epoch 293 - loss: 5.47619104385376
Epoch 294 - loss: 5.47619104385376
Epoch 295 - loss: 5.47619104385376
Epoch 296 - loss: 5.476190567016602
Epoch 297 - loss: 5.47619104385376
Epoch 298 - loss: 5.47619104385376
Epoch 299 - loss: 5.47619104385376
```

```
when x = tensor([1., 3.]), y = tensor([3.5714], grad_fn=<SqueezeBackward4>)
when x = tensor([2., 6.]), y = tensor([7.1429], grad_fn=<SqueezeBackward4>)
when x = tensor([3., 9.]), y = tensor([10.7143], grad_fn=<SqueezeBackward4>)
```

[1]:
```
[<matplotlib.lines.Line2D at 0x11091e8a0>]
```



0.1 Stochastic gradient descent

Both models start from weights `[1, -1]` and train for 300 epochs with learning rate 0.01 and momentum 0.5. SGD shuffles the points with seed 0 and updates after each point. Both comparison curves show full-data MSE after each epoch.

[2]:
```
batch_net = net
with torch.no_grad():
    batch_final_mse = criterion(batch_net(data), target).item()
batch_hist = [value.item() for value in hist[1:]] + [batch_final_mse]

sgd_net = Net()
sgd_net.fc1.weight = nn.Parameter(torch.tensor([[1., -1.]]))
sgd_optimizer = optim.SGD(sgd_net.parameters(), lr=0.01, momentum=0.5)
shuffle_generator = torch.Generator().manual_seed(0)
sgd_hist = []
sgd_updates = 0

for epoch in range(300):
    for index in torch.randperm(len(data), generator=shuffle_generator).tolist():
        X = data[index:index + 1]
        Y = target[index:index + 1]
        sgd_optimizer.zero_grad()
        outputs = sgd_net(X)
```

```
        loss = criterion(outputs, Y)
        loss.backward()
        sgd_optimizer.step()
        sgd_updates += 1
    with torch.no_grad():
        epoch_mse = criterion(sgd_net(data), target).item()
    sgd_hist.append(epoch_mse)
    if epoch == 0 or (epoch + 1) % 50 == 0:
        print("SGD epoch {} - full-data MSE: {:.6f}".format(epoch + 1, epoch_mse))

assert sgd_updates == 300 * len(data)
print("SGD updates:", sgd_updates)
```

```
SGD epoch 1 - full-data MSE: 50.978275
SGD epoch 50 - full-data MSE: 59.053577
SGD epoch 100 - full-data MSE: 45.657497
SGD epoch 150 - full-data MSE: 98.223198
SGD epoch 200 - full-data MSE: 10.239625
SGD epoch 250 - full-data MSE: 22.272635
SGD epoch 300 - full-data MSE: 5.776175
SGD updates: 900
```

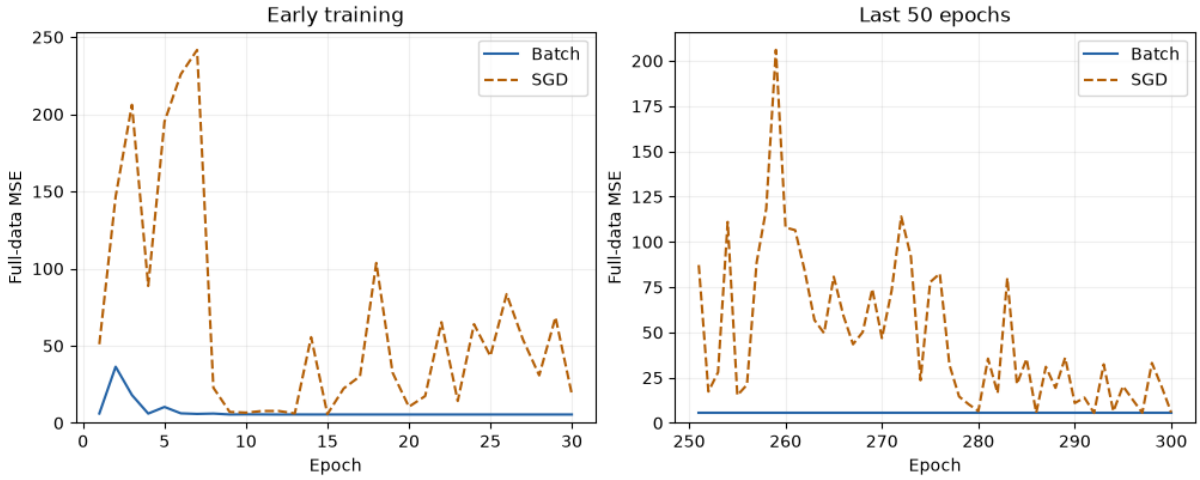
[3]:
```
with torch.no_grad():
    batch_predictions = batch_net(data)
    sgd_predictions = sgd_net(data)
    sgd_final_mse = criterion(sgd_predictions, target).item()

print("Method     Updates    Final full-data MSE")
print("Batch      {:7d}    {:.6f}".format(300, batch_final_mse))
print("SGD        {:7d}    {:.6f}".format(sgd_updates, sgd_final_mse))
print("\n x1  x2  Target    Batch       SGD")
for row, y, batch_y, sgd_y in zip(data, target, batch_predictions, sgd_predictions):
    print("{:3.0f} {:3.0f} {:7.2f} {:9.4f} {:9.4f}".format(
        row[0].item(), row[1].item(), y.item(), batch_y.item(), sgd_y.item()))
print("Last 50 epoch MSE range: batch [{:.6f}, {:.6f}]; SGD [{:.6f}, {:.6f}]".format(
    min(batch_hist[-50:]), max(batch_hist[-50:]),
    min(sgd_hist[-50:]), max(sgd_hist[-50:])))

fig, axes = plt.subplots(1, 2, figsize=(10, 4), layout="constrained")
axes[0].plot(range(1, 31), batch_hist[:30], label="Batch", color="#2463a6")
axes[0].plot(range(1, 31), sgd_hist[:30], label="SGD", color="#b45f06", linestyle="--")
axes[0].set(title="Early training", xlabel="Epoch", ylabel="Full-data MSE")
axes[1].plot(range(251, 301), batch_hist[-50:], label="Batch", color="#2463a6")
axes[1].plot(range(251, 301), sgd_hist[-50:], label="SGD", color="#b45f06", linestyle="--")
axes[1].set(title="Last 50 epochs", xlabel="Epoch", ylabel="Full-data MSE")
for ax in axes:
    ax.set_ylim(bottom=0)
    ax.grid(True, alpha=0.2)
    ax.legend()
plt.show()
```

```
Method     Updates    Final full-data MSE
Batch          300    5.476191
SGD            900    5.776175

 x1  x2  Target    Batch       SGD
  1   3    1.00    3.5714    3.8250
  2   6    5.00    7.1429    7.6499
  3   9   13.00   10.7143   11.4749
Last 50 epoch MSE range: batch [5.476191, 5.476191]; SGD [5.522751, 206.057373]
```



0.2 Why the curves differ

Each data point pushes the model toward a different solution. Because $x_2 = 3x_1$, the prediction is effectively

$$\hat{y} = (w_1 + 3w_2)x_1 = cx_1.$$

The three samples want $c = 1$, $c = 2.5$, and $c = 13/3 \approx 4.33$. One slope cannot satisfy all three.

Batch training averages these conflicting gradients before updating, so it settles at the best compromise. Its loss oscillates initially, then settles near 5.476191.

SGD updates using one sample at a time. An update that improves one sample can worsen the others, causing the full-dataset loss to bounce around. Shuffling changes which sample pushes the model next. Its final MSE is 5.776175 in this run, but that value is only one point among its ongoing fluctuations.

The learning rate of **0.01** stays constant, and **momentum of 0.5** carries previous updates forward, which can amplify overshooting. Larger inputs also produce larger gradients: for a sample with input vector x, the gradient is $2(\hat{y} - y)x$. Together, these explain the pronounced fluctuations.

Reducing or gradually decaying SGD's learning rate would help it settle closer to the batch solution.

The least-squares compromise is

$$c = \frac{1 \cdot 1 + 2 \cdot 5 + 3 \cdot 13}{1^2 + 2^2 + 3^2} = \frac{25}{7},$$

with minimum full-data MSE $115/21 \approx 5.476190$. Thus neither method can achieve zero loss with this model. Both methods train for 300 epochs, but SGD performs 900 updates versus 300 batch updates. Every plotted loss is evaluated on the full dataset, so the observed noise reflects changes in the model rather than simply switching which sample's loss is plotted.

[ ]:

[ ]:

Question 6: Batch Training versus SGD

Observed behavior and explanation

Training conditions

Both models start from weights $[1, -1]$ and train for 300 epochs with learning rate 0.01 and momentum 0.5. Batch training updates once per epoch; SGD updates after each point, with shuffled order using seed 0. This gives 300 batch updates and 900 SGD updates. Both comparison curves show full-data MSE after each epoch.

Observed behavior

With the same initialization and optimizer settings, batch training oscillates initially, then settles near the minimum full-data MSE. Per-point SGD has a much noisier curve, including large fluctuations late in training.

Method	Parameter updates	Final full-data MSE
Batch gradient descent	300	5.476191
Per-point SGD	900	5.776175

SGD's final loss is one point among ongoing fluctuations, rather than evidence that its weights have settled. During the last 50 epochs, its MSE ranged from approximately 5.523 to 206.057, whereas the batch MSE remained approximately 5.476191.

Why this happens

Each sample pushes the weights toward a different solution. Here $x_2 = 3x_1$, so the model is effectively

$$\hat{y} = w_1 x_1 + w_2 x_2 = (w_1 + 3w_2)x_1 = cx_1.$$

The three input–target pairs require slopes 1, 5/2, and 13/3. No single slope fits all three exactly. Batch training averages their gradients before each update, allowing it to settle at the best compromise. SGD uses one sample's gradient, so improving that sample can worsen the other two. Shuffling changes the order of these competing pushes.

The constant learning rate keeps SGD's updates from shrinking automatically. Momentum carries earlier updates forward and can amplify overshooting. Larger inputs also create larger gradients, since the sample gradient is $2(\hat{y} - y)x$. Thus the loss fluctuations are caused by changes in the weights, not merely by plotting a different sample's loss at each step.

The least-squares compromise is

$$c = \frac{1(1) + 2(5) + 3(13)}{1^2 + 2^2 + 3^2} = \frac{25}{7}, \qquad L_{\min} = \frac{115}{21} \approx 5.476190.$$

Reducing or decaying SGD's learning rate would help it settle closer to this compromise. It would not eliminate the model's inability to fit all three targets.